Hands-on Labs

Afternoon Session

Day 3 - Afternoon Labs

KavinSchool

Lab 3.2: Build a RAG Chatbot

End-to-End Implementation

KavinSchool

Lab 3.2: End-to-End RAG Chatbot

Goal: Build a complete RAG system that answers questions about a knowledge base.

Steps:

1. Load markdown docs from `day3/knowledge-base/`
2. Chunk the documents (with metadata)
3. Embed chunks and store in FAISS
4. Build a query loop: user asks, system retrieves + generates
5. Cite sources in every answer

Time: 45 minutes | **File:** `day3/labs/lab3_2_rag_chatbot.py`

The Knowledge Base

The lab retrieves answers from `day3/afternoon-labs/knowledge-base/`, a small set of Python reference docs:

File	Topic
`python-data-structures.md`	Lists, dicts, sets, tuples
`python-functions.md`	Defining and calling functions
`python-error-handling.md`	try/except, exceptions
`python-testing.md`	Writing and running tests
`python-virtual-environments.md`	venv, dependency isolation

The Knowledge Base

Why these files?

- Real markdown docs give the chatbot a **factual source** to ground answers in
- Small and familiar: you can verify the bot quotes the docs, not its training data
- Multiple files let you test **source citation** across documents

> This is the "your data" in RAG. Swap in your own docs later and the same pipeline works.

Lab 3.2: Expected Interaction

```
🔍 RAG Chatbot — Ask questions about the knowledge base
Type 'quit' to exit.

You: What Python data structures should I use for key-value pairs?

Bot: Based on the documentation, you should use Python dictionaries
     for key-value pairs. Dictionaries provide O(1) average lookup
     time and support any hashable type as keys.
     [Source: python-data-structures.md]

     For ordered key-value pairs, consider using
     collections.OrderedDict (Python 3.6+ dicts maintain insertion
     order by default). [Source: python-data-structures.md]

Sources used: python-data-structures.md
```

Lab 3.3: Flask API for the RAG Chatbot

Integration & Channels

KavinSchool

Lab 3.3: Flask API

Goal: Wrap your Lab 3.2 RAG chatbot in a Flask API endpoint.

Requirements:

1. `POST /api/chat` — accepts `{"message": "..."}`, returns `{"answer": "...", "sources": [...]}`
2. `GET /api/health` — returns `{"status": "ok", "chunks_loaded": N}`
3. Basic error handling (missing message, API failures)
4. Test with `curl` or Python `requests`

Lab 3.3: Flask API

Stretch goals:

- Add session-based conversation history
- Add response caching for repeated queries
- Add a `GET /api/sources` endpoint listing all indexed documents

Time: 20 minutes | **File:** `day3/labs/lab3_3_flask_api.py`